

34 — LLM Output Guardrails

What it is

LLM output guardrails are a set of validation and retry mechanisms that sit between every LLM call and the rest of your application. They solve a fundamental problem: LLMs are non-deterministic text generators, not reliable APIs. Even with detailed instructions, an LLM might return invalid JSON, forget required fields, wrap its response in markdown code fences, or produce content that sounds authoritative but is made up.

Guardrails address this at three levels:

1. **Structural validation** — After each LLM call, check that the output is valid JSON, contains all required fields, and that each field has the correct type (string, list, etc.). If it fails, retry the request with corrective feedback that tells the model exactly what went wrong.
2. **Content safety checks for RAG answers** — When the LLM generates an answer from retrieved documents, verify that it actually cites those documents (e.g. [1] , [2] references), is not suspiciously short or long, and does not contain "hallucination markers" — phrases like "As an AI language model" that signal the LLM ignored the context and fell back to its training data.
3. **Translation sanity checks** — After every translation, verify the output is not empty, not identical to the input (which would mean no translation happened), and not suspiciously shorter or longer than the original (which could indicate truncation or hallucinated expansion).

Every validation failure is logged with an `EVIDENCE_GUARDRAIL_*` marker so it shows up in UAT evidence.

Real-life analogy

Think of a quality inspector on a factory assembly line. Each product (LLM output) rolls down the belt and the inspector checks it against a spec sheet:

- **Does it have all the required parts?** (required fields present)
- **Are the parts the right type?** (string where string is expected, list where list is expected)

- **Is the packaging correct?** (valid JSON, no markdown code fences wrapping it)

If a product fails inspection, it does not get shipped. Instead, the inspector sends it back to the workstation with a note stapled to it: "Missing the status label. The `add_words` compartment has a string instead of a list." The worker (the LLM) gets a second chance to fix it. If it fails again after the maximum retries, the product is set aside with a "REVIEW" status and flagged for manual attention.

For translations, the inspector also does a basic sanity check: if the "translated" product looks identical to the original, or is suspiciously tiny compared to the input, something went wrong. For RAG answers, the inspector checks that the answer actually references the source documents it was given — an answer that ignores its evidence is suspicious.

Why we used it here

Our ETL pipeline has three places where LLM output quality directly affects data integrity:

1. Enrichment (R5) — Each category and company record gets AI-generated metadata: `ai_type`, `status`, `add_words` (SEO keywords), `recommended_headline`, `synthetic_questions`, and `topic_suggestions`. All six fields are required and have specific types. Without guardrails, a single malformed response (missing `add_words`, or returning `"add_words": "keyword1, keyword2"` as a string instead of a list) would either crash downstream code or silently store bad data.

2. Translation (R3) — Every record needs both Hungarian and English text. If a translation call returns an empty string, or returns the input unchanged, we end up with missing or duplicate-language data. The pipeline would not crash — it would just write bad rows that look correct until someone searches for them and gets no results.

3. RAG answers — The search API generates natural-language answers from retrieved documents. If the LLM ignores the context and generates from its training data (hallucination), users get authoritative-sounding wrong answers. If it forgets to cite sources, users cannot verify the claims.

The non-deterministic nature of LLMs makes these failures intermittent — they work 95% of the time, which makes bugs hard to catch in testing. Guardrails turn silent data corruption into logged, actionable warnings.

Code walkthrough

`bilingual_etl/providers/guardrails.py`

This module is the central guardrails implementation. It has four main functions:

`strip_code_fences()` — preprocessing

```
_CODE_FENCE_PATTERN = re.compile(r"^```(?:json)?\s*|\s*```$", re.IGNORECASE)

def strip_code_fences(raw: str) -> str:
    return _CODE_FENCE_PATTERN.sub("", raw).strip()
```

Many LLMs wrap JSON in markdown code fences (````json ... ````) even when told not to. This strips them before JSON parsing. It runs on every response, not conditionally — defensive by default.

`validate_json_response()` — structural validation

```
def validate_json_response(
    raw: str,
    required_fields: list[str],
    field_types: dict[str, type] | None = None,
) -> tuple[dict | None, list[str]]:
```

Three layers of validation in sequence:

1. **Parse** — `json.loads()` on the fence-stripped text. If it fails, return immediately with an "Invalid JSON" error.
2. **Type check root** — Ensure the result is a dict (JSON object), not a list or string.
3. **Required fields** — Check every field in `required_fields` exists in the dict.
4. **Field types** — If `field_types` is provided, check each present field has the expected Python type.

Returns a tuple of `(parsed_data, errors)`. If `errors` is empty, the data is valid.

`generate_with_guardrails()` — retry loop with corrective feedback

```
def generate_with_guardrails(
    llm, prompt, system_prompt,
    required_fields, field_types=None,
```

```
max_retries=MAX_GUARDRAIL_RETRIES, # default: 2
) -> tuple[dict | None, list[str]]:
```

This is the key function. It wraps the generate-then-validate cycle in a retry loop:

1. Call the LLM via `generate_with_retry()` (which handles rate limits and transient API errors — a separate concern).
2. Validate the response with `validate_json_response()`.
3. If valid, return immediately. If this was a retry, log `EVIDENCE_GUARDRAIL_RETRY_SUCCESS`.
4. If invalid and retries remain, **append corrective feedback** to the prompt:

Your previous response had validation errors:

- Missing required field: add words
- Field 'status' should be str, got int

Please fix these issues and respond with valid JSON containing all required fields: `ai_type`, `status`, `add_words`, ...

1. Retry with the augmented prompt. The LLM now sees its own mistakes and the expected format.
2. If all retries exhausted, log `EVIDENCE_GUARDRAIL_EXHAUSTED` and return whatever partial data exists along with the accumulated errors.

The total attempts are `max_retries + 1` (1 initial + 2 retries by default = 3 attempts).

`validate_rag_answer()` — RAG content checks

```
def validate_rag_answer(answer: str, citations: list[dict]) -> list[s
```

Four checks on the generated answer:

Check	What it catches
<code>len(answer.strip()) < 10</code>	LLM returned empty or near-empty answer
No <code>[N]</code> pattern when citations exist	LLM ignored the source documents
Hallucination marker phrases	LLM fell back to training data instead of context
<code>len(answer) > 5000</code>	

Check	What it catches
	Runaway generation (possible loop or prompt leak)

Returns a list of warning strings (empty = all clear). These are warnings, not hard failures — the answer is still returned to the user, but with `guardrail_warnings` attached so the caller knows the quality is suspect.

`validate_translation()` — translation sanity checks

```
def validate_translation(original, translated, source_lang, target_la
```

Four checks:

Check	What it catches
Empty or whitespace-only	Translation call returned nothing
Identical to original	LLM echoed the input instead of translating
Length < 20% of original	Severe truncation
Length > 500% of original	Hallucinated expansion

Like RAG validation, these are warnings logged with `EVIDENCE_GUARDRAIL_TRANSLATION_WARNINGS`.

How `ai_enrichment.py` uses guardrails

Before guardrails, the enrichment module had its own inline JSON parsing with `strip_code_fences()` and `json.loads()`, no retry, and no field validation. Now it delegates entirely:

```
from bilingual_etl.providers.guardrails import generate_with_guardrai

REQUIRED_FIELDS = ["ai type", "status", "add words",
                   "recommended_headline", "synthetic_questions", "top

FIELD TYPES = {
    "ai type": str, "status": str, "add words": list,
    "recommended_headline": str, "synthetic_questions": list, "topic_s
}
```

```
def enrich(text, llm=None, prompt_store=None):
    # ...
    data, errors = generate_with_guardrails(
        llm, text, system_prompt=system_prompt,
        required_fields=REQUIRED_FIELDS, field_types=FIELD_TYPES,
    )

    if errors or data is None:
        logger.warning(f"Enrichment guardrail fallback: {errors}")
        return _empty_enrichment() # safe default with status="REVIEW"

    result = {field: data.get(field) for field in REQUIRED_FIELDS}
    return result
```

Key design choices: - `REQUIRED_FIELDS` and `FIELD_TYPES` are declared as module-level constants — they document the enrichment contract in one place. - On guardrail failure (after all retries), the function returns `_empty_enrichment()` with `status="REVIEW"` rather than crashing. This means the ETL run completes, and the bad record is flagged for human review.

How `translator.py` uses guardrails

Translation validation is simpler — translations are plain text, not JSON, so there is no structural validation. Instead, `validate_translation()` is called after the markdown-stripping step:

```
result = _strip_markdown_emphasis(generate_with_retry(llm, prompt, sy

from bilingual etl.providers.guardrails import validate_translation
validate_translation(text, result, source_language, target_language)
```

The warnings are logged but the translation is still returned. This is a conscious choice: a suspiciously short translation is probably still better than no translation, and the `EVIDENCE_GUARDRAIL_TRANSLATION_WARNINGS` log lets operators investigate after the run.

How `search_api/services/rag.py` uses guardrails

After the LLM generates an answer from retrieved documents:

```
from bilingual etl.providers.guardrails import validate_rag_answer
guardrail_warnings = validate_rag_answer(answer, citations)

return {
```

```
"answer": answer,  
"citations": citations,  
"guardrail warnings": guardrail_warnings,  
# ... other fields  
}
```

The `guardrail_warnings` list is included in the API response. The frontend or API consumer can:

- Display a "this answer may not be fully supported by sources" notice if warnings are present.
- Filter or flag answers with hallucination markers.
- Log warning frequency for monitoring LLM quality over time.

How to explain this in an interview / to a teammate

"LLMs are non-deterministic — even with the same prompt, they sometimes return invalid JSON, forget required fields, or wrap output in markdown. In our enrichment pipeline, we need structured JSON with six specific fields for every record, so we built a guardrails layer that validates every LLM response against a schema, then retries with corrective feedback if it fails — basically telling the model 'you missed these fields, try again.' For RAG answers, we check that the response actually cites the source documents and does not contain hallucination markers. For translations, we verify the output is not empty or identical to the input. Every failure is logged with evidence markers, so we can track LLM reliability across runs."